Laboratorium Sztucznej Inteligencji

Analiza dyskryminacyjna

Mateusz Braszczok, Daniel Kopiecki

21 grudnia 2022



Politechnika Śląska

Contents

1 Zadanie 1 **3**

2 Zadanie 2 **3**
2.1 Liniowa analiza dyskryminacyjna 3
2.2 Diagonalna liniowa analiza dyskryminacyjna 4
2.3 Porównanie czasu działania 4

3 Zadanie 3 **5**

4 Kod źródłowy **7**

List of Tables

List of Figures

1 Wygenerowane dane 5
2 Przykładowy proces klasyfikacji (metoda LDA) 5
3 Przebieg klasyfikacji metodą QDA 6

1 Zadanie 1

Z metod analizy dyskryminacyjnej będących tematem ćwiczenia wraz z założeniami, które je opisują wyróżniamy:

1. Liniowa analiza dyskryminacyjna (ang. *linear discriminant analysis*, LDA) - głównym zadaniem jest znalezienie kierunku w przestrzeni cech, który najlepiej rozdziela obie klasy. Zakłada, że funkcja gęstości prawdopodobieństwa obu klas ma rozkład normalny i ich macierze kowariancji są identyczne.

2. Diagonalna liniowa analiza dyskryminacyjna (ang. *diagonal linear discriminant analysis*, DLDA) - jest uproszczonym wariantem metody LDA. Poza równością macierzy kowariancji zakłada również brak korelacji pomiędzy cechami. Wszystkie elementy poza przekątną macierzy kowariancji są zerowe.

3. Kwadratowa analiza dyskryminacyjna (ang. *quadtratic discriminant analysis*, QDA) - jest rozwinięciem metody LDA i podobnie jak ona zakłada, że funkcja gęstości prawdopodobieństwa obu klas ma rozkład normalny, lecz w odróżnieniu w jej przypadku założenie o równości macierzy kowariancji nie jest wymagane. Oznacza to, że macierze kowariancji są estymowane osobno.

2 Zadanie 2

Celem zadania jest implementacja oraz porównanie złożoności obliczeniowej metod LDA i DLDA. Obydwie metody zostały zaimplementowane w oparciu o środowisko MATLAB i pozwalają na obsłużenie danych wejściowych o dowolnej liczbie cech.

2.1 Liniowa analiza dyskryminacyjna

Implementacja polega na zaprogramowaniu wzoru na następującą funkcję klasyfikującą:

$$f(x) = (\bar{x}_1 - \bar{x}_2)^T W^{-1}(x - \frac{1}{2}(\bar{x}_1 + \bar{x}_2)) \tag{1}$$

gdzie:

- $f(x)$ - wyjście klasyfikatora
- $\bar{x}_i$ - średnia cech i-tej klasy
- W - macierz kowariancji
- x - próbka testowa

Zaimplementowana funkcja jako argumenty przyjmuje testowaną próbkę, średnią cech pierwszej klasy, średnią cech drugiej klasy oraz macierz kowariancji. Każdy człon **??** jest przeliczany osobno dla lepszej czytelności.

Implementację przedstawiono na poniższym listingu.

```
1 function classification = lda(obj, sample, firstMean, secondMean,
      covarianceMatrix)
2     meanDifference = (firstMean - secondMean);
```

```
    inverseCovarianceMatrix = inv(covarianceMatrix);
    classification = meanDifference * inverseCovarianceMatrix;
    part = (sample - 0.5 * (firstMean + secondMean)');
    classification = classification * part;
end
```

2.2 Diagonalna liniowa analiza dyskryminacyjna

Zaimplementowana funkcja w tym przypadku praktycznie nie różni się od liniowej analizy dyskryminacyjnej. Jak sama nazwa wskazuje, macierz kowariancji, która wchodzi na wejście funkcji, musi być diagonalna, więc metoda zakłada brak korelacji pomiędzy cechami. Powoduje to możliwość uniknięcia czasochłonnego procesu odwracania macierzy na rzecz prostej zależności:

$$A = \begin{bmatrix} a_1 & & \\ & \ddots & \\ & & a_r \end{bmatrix} \implies A^{-1} = \begin{bmatrix} a_1^{-1} & & \\ & \ddots & \\ & & a_r^{-1} \end{bmatrix} \tag{2}$$

Implementację przedstawiono na poniższym listingu.

```
function classification = dlda(obj, sample, firstMean, secondMean
    , invCovarianceMatrix)
    meanDifference = (firstMean - secondMean);
    classification = meanDifference * (invCovarianceMatrix);
    part = (sample - 0.5 * (firstMean + secondMean)');
    classification = classification * part;
end
```

2.3 Porównanie czasu działania

Jak opisano wyżej, pomimo, że algorytmy różnią się nieznacznie, to proces odwracania macierzy może mieć znaczy wpływ na złożoność obliczeniową. Dla klasycznego przypadku odwracanie macierzy n na n skutkuje złożonością obliczeniową $O(n^3)$, gdzie w przypadku macierzy diagonalnej sprowadza się to tylko do n dzieleń elementów na przekątnej.

W celu porównania czasu działania algorytmów, przeprowadzono 50 eksperymentów klasyfikacji stu próbek testowych i policzono średnią z czasów działania poszczególnych algorytmów. Wyniki są następujące:

Metoda	Średni czas działania $[s]$
LDA	0.001127
DLDA	0.000542

Jak można zaobserwować z powyższej tabeli, hipoteza autorów się sprawdziła i czas działania algorytmu DLDA jest w przybliżeniu dwa razy szybszy niż LDA.

3 Zadanie 3

Wygenerowano syntetyczny zbiór składający się z 200 próbek, spośród których można rozróżnić dwie klasy abstrakcji, a każda próbka ma zestaw dwóch cech. Klasy A1 i A2 zostały wygenerowane z rozkładu normalnego. Warto wspomnieć, że dane dla określonych metod muszą być wygenerowane w określony sposób. W każdej metodzie muszą pochodzić z rozkładu normalnego, gdzie dodatkowo dla LDA i DLDA klasy muszą mieć równe macierze kowariancji.

Przykładową generację z równymi macierzami kowariancji przedstawiono na rysunku poniżej.

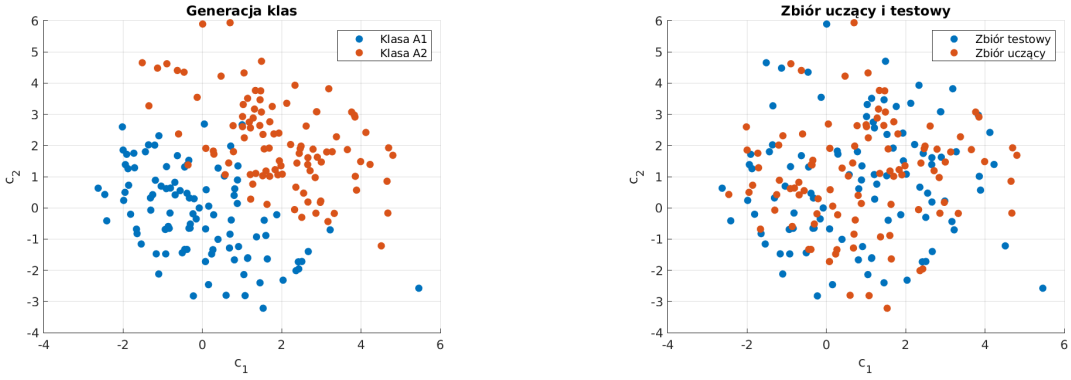


Figure 1: Wygenerowane dane

Kolejną częścią ćwiczenia było porównanie dokładności klasyfikacji analizowanych metod. W tym celu przeprowadzono 50 eksperymentów klasyfikacji dla wszystkich metod, dla każdej eksperymentu policzono wskaźnik ACC dany zależnością:

$$ACC = \frac{TP + TN}{TP + FP + TN + FN} \tag{3}$$

i następnie policzono średnią. Przykładowe procesy klasyfikacji przedstawiono na poniższych rysunkach.

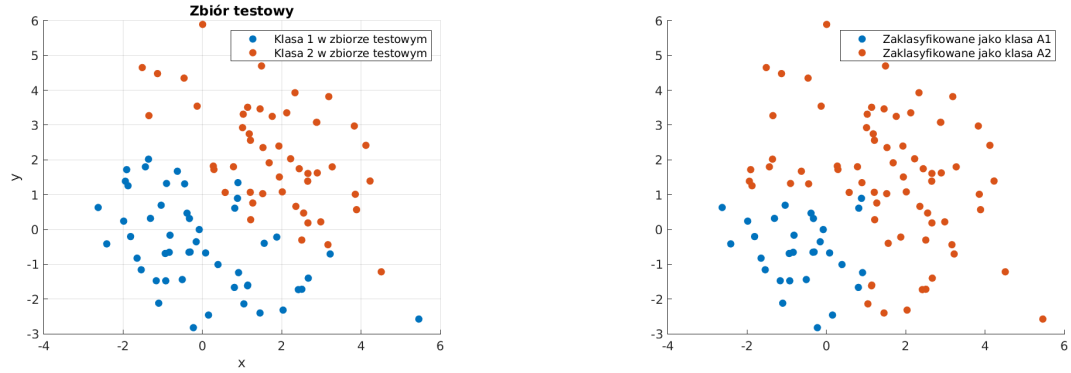


Figure 2: Przykładowy proces klasyfikacji (metoda LDA)

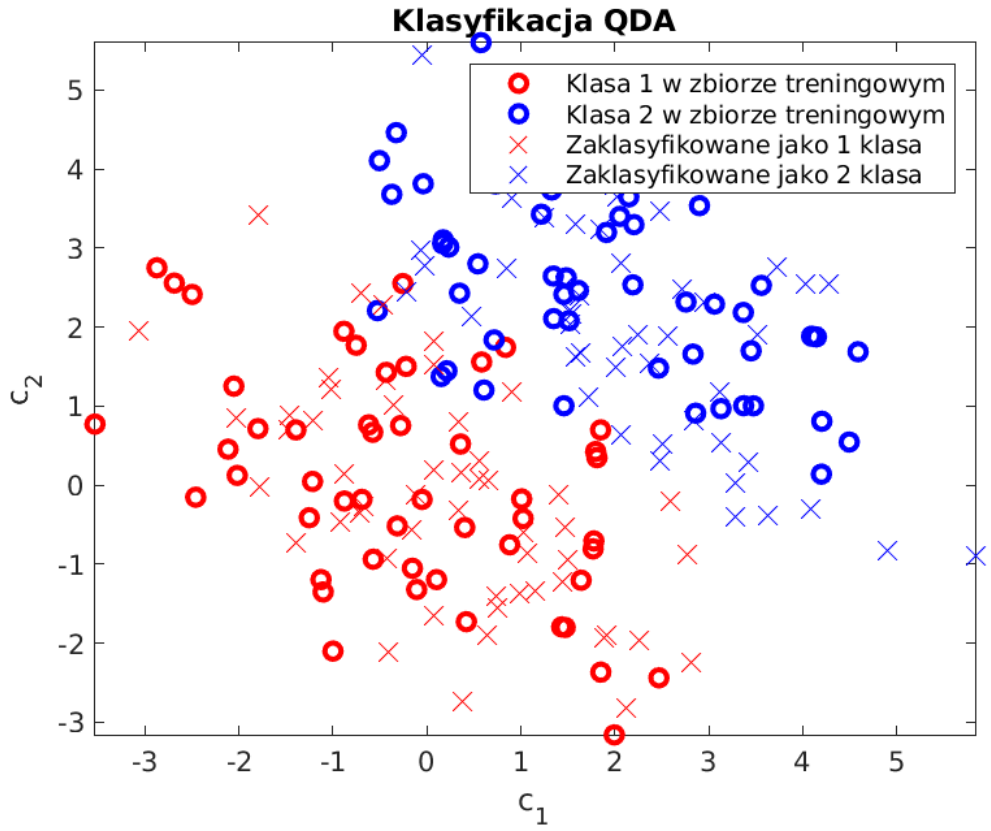


Figure 3: Przebieg klasyfikacji metodą QDA

Obliczone średnie wskaźników ACC przedstawiono na poniższej tabeli: Ostatnią częścią

Metoda	$\bar{ACC}$, %
DLDA	75.2800
LDA	76.5400
QDA	91.6

ćwiczenia było odpowiedzenie na pytanie, jak analizowane metody działają, kiedy dane wejściowe nie spełniają ich założeń. W przypadku metody QDA, równość macierzy kowariancji nie wpływa na jakość klasyfikacji, dlatego jest to metoda najbardziej uniwersalna. W przypadku metody LDA, kiedy macierze kowariancji nie są równe, wskaźnik ACC spada. Dla następujących macierzy kowariancji:

$$W_1 = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}, \quad W_2 = \begin{bmatrix} 5 & -1 \\ -1 & 5 \end{bmatrix} \tag{4}$$

średni wskaźnik ACC spadł o 2.7%. Stopniowo zmniejszając podobieństwo macierzy W_1 i W_2, spadał jeszcze mocniej. Analogicznie było w przypadku metody DLDA, tylko tam jeszcze rolę odgrywała zerowość elementów poza główną przekątną i ich zmiana dawała również analogiczne efekty (spadek jakości klasyfikacji).

Z wniosków na aktualną chwilę można wyciągnać wniosek, że najlepsze rezultaty daje metoda QDA. W jej przypadku dokładność klasyfikacji była największa i wynosiła aż

91.6%. Klasyfikację tą metodą przeprowadzano w oparciu o funkcję *classify* i jej implementacja była następująca:

```
function C = qda(obj, testSet, trainingSet)
    testSet = testSet(:, 1 : 2);
    group = trainingSet(:, 3);
    trainingSet = trainingSet(:, 1 : 2);

    [C, err, poserior, logp, coeff] = classify(testSet,
       trainingSet, group, 'quadratic');

    K = coeff(1,2).const;
    L = coeff(1,2).linear;
    Q = coeff(1,2).quadratic;

    gcf = figure("visible", "off");
    X = testSet(:, 1);
    Y = testSet(:, 2);
    h1 = gscatter(trainingSet(:, 1), trainingSet(:, 2), group,'rb
       ', 'o', [], 'off');
    h1(1).LineWidth = 2;
    h1(2).LineWidth = 2;
    hold on;
    h2 = gscatter(X, Y, C, 'rb', 'x', 10, 'off');
    hold off;
    axis tight
    xlabel('c_1')
    ylabel('c_2')
    title('Klasyfikacja QDA')
    legend("Klasa 1 w zbiorze treningowym", "Klasa 2 w zbiorze
       treningowym", "Zaklasyfikowane jako 1 klasa", "
       Zaklasyfikowane jako 2 klasa")
    saveas(gcf, "qda.png");
end
```

Warto wspomnieć, że metoda QDA poza najlepszą dokładnością ma też najdłuższy czas działania. W jej przypadku średni czas dla 50 klasyfikacji wyniósł aż 0.408626 s.

4 Kod źródłowy

```
classdef SP2 < handle
    properties
        A1_TEST_SET = [1, 4; 1, 1; 4, 4; 4, 1;]
        A2_TEST_SET = [3, 6; 6, 6; 3, 3; 6, 3;]

        A1_SET
        A2_SET

        LDA = 0
        DLDA = 1
```

```
        QDA = 2

        N = 100
    end

    methods (Access = private)
        function classification = lda(obj, sample, firstMean,
            secondMean, covarianceMatrix)
            meanDifference = (firstMean - secondMean);
            inverseCovarianceMatrix = inv(covarianceMatrix);
            classification = meanDifference *
                inverseCovarianceMatrix;
            part = (sample - 0.5 * (firstMean + secondMean)');
            classification = classification * part;
        end

        function classification = dlda(obj, sample, firstMean,
            secondMean, invCovarianceMatrix)
            meanDifference = (firstMean - secondMean);
            classification = meanDifference * (
                invCovarianceMatrix);
            part = (sample - 0.5 * (firstMean + secondMean)');
            classification = classification * part;
        end

        function C = qda(obj, testSet, trainingSet)
            tic
            testSet = testSet(:, 1 : 2);
            group = trainingSet(:, 3);
            trainingSet = trainingSet(:, 1 : 2);

            [C, err, poserior, logp, coeff] = classify(testSet,
                trainingSet, group, 'quadratic');

            K = coeff(1,2).const;
            L = coeff(1,2).linear;
            Q = coeff(1,2).quadratic;

            gcf = figure("visible", "off");
            X = testSet(:, 1);
            Y = testSet(:, 2);
            h1 = gscatter(trainingSet(:, 1), trainingSet(:, 2),
                group,'rb', 'o', [], 'off');
            h1(1).LineWidth = 2;
            h1(2).LineWidth = 2;
            hold on;
            h2 = gscatter(X, Y, C, 'rb', 'x', 10, 'off');
            hold off;
            axis tight
```

```
54              xlabel('c_1')
55              ylabel('c_2')
56              title('Klasyfikacja QDA')
57              legend("Klasa 1 w zbiorze treningowym", "Klasa 2 w
                    zbiorze treningowym", "Zaklasyfikowane jako 1
                    klasa", "Zaklasyfikowane jako 2 klasa")
58              saveas(gcf, "qda.png");
59              toc
60          end
61      end
62
63      methods (Access = public)
64          function myACC = test(obj, testSet, trainingSet, type)
65
66              [testA1, testA2] = obj.getClasses(testSet);
67              [a1Set, a2Set] = obj.getClasses(trainingSet);
68              gcf = figure("visible", "off");
69              scatter(testA1(:, 1), testA1(:, 2), 'fill');
70              hold on;
71              scatter(testA2(:, 1), testA2(:, 2), 'fill');
72              grid on;
73              xlabel("x");
74              ylabel("y");
75              title("Zbior testowy");
76              legend("Klasa 1 w zbiorze testowym", "Klasa 2 w
                    zbiorze testowym");
77              saveas(gcf, "images/test.png");
78
79              A1Mean = mean(a1Set);
80              scatter(A1Mean(1), A1Mean(2), 'x')
81              A2Mean = mean(a2Set);
82              scatter(A2Mean(1), A2Mean(2), 'x')
83
84              A1Cov = cov(a1Set);
85              A2Cov = cov(a2Set);
86
87              Cov = A1Cov;
88              InvCov = [];
89              if (type == obj.DLDA)
90              for i = 1 : length(Cov)
91                  for j = 1 : length(Cov)
92                      if i ~= j
93                          InvCov(i, j) = 0;
94                      else
95                          InvCov(i, j) = 1 / Cov(i, j);
96                      end
97                  end
98              end
99              end
```

```
100
101             classification = 0;
102             j = 1;
103             k = 1;
104             A1_class = [];
105             A2_class = [];
106             acc = ACC();
107
108             if (type == obj.LDA || type == obj.DLDA)
109                 tic
110                 for i = 1 : length(testSet)
111                     testPoint = testSet(i, 1 : 2);
112                     if (type == obj.LDA)
113                         classification = obj.lda(testPoint,
                                A1Mean, A2Mean, Cov);
114                     elseif (type == obj.DLDA)
115                         classification = obj.dlda(testPoint,
                                A1Mean, A2Mean, Cov);
116                     end
117                     if (classification > 0)
118                         A1_class(j, 1 : 2) = testPoint;
119                         j = j + 1;
120                     else
121                         A2_class(k, 1 : 2) = testPoint;
122                         k = k + 1;
123                     end
124
125                     acc.calculate(classification, testSet(i, :))
126                 end
127                 toc
128                 gcf = figure("visible", "off");
129                 scatter(A1_class(:, 1), A1_class(:, 2), 'fill');
130                 hold on;
131                 scatter(A2_class(:, 1), A2_class(:, 2), 'fill');
132                 hold off;
133                 legend(["Zaklasyfikowane jako klasa A1", "
                        Zaklasyfikowane jako klasa A2"])
134                 saveas(gcf, "images/class.png");
135             else
136                 classification = obj.qda(testSet, trainingSet)
137                 for i = 1 : length(classification)
138                     acc.calculateQDA(classification(i), testSet(i
                            , :))
139                 end
140             end
141             myACC = acc.acc
142         end
143
```

```
144         function [testSet, trainingSet] = generateData(obj, muA1,
                sigmaA1, muA2, sigmaA2)
145             A1 = mvnrnd(muA1, sigmaA1, obj.N);
146             A2 = mvnrnd(muA2, sigmaA2, obj.N);
147 
148             A1 = A1 * inv(chol(cov(A1)));
149             A1 = A1 * chol(sigmaA1);
150             c = cov(A1)
151 
152             A2 = A2 * inv(chol(cov(A2)));
153             A2 = A2 * chol(sigmaA2);
154             c = cov(A2)
155 
156             gcf = figure("visible", "off");
157             scatter(A1(:, 1), A1(:, 2), 'fill');
158             grid on;
159             hold on;
160             scatter(A2(:, 1), A2(:, 2), 'fill');
161             title("Generacja klas");
162             xlabel("c_1");
163             ylabel("c_2");
164             legend("Klasa A1", "Klasa A2");
165             saveas(gcf, "images/a1_a2.png")
166 
167             testSet = [];
168             trainingSet = [];
169 
170             testSet(1:50, 1:2) = A1(1:50, :);
171             testSet(1:50, 3) = ones(50, 1);
172             trainingSet(1:50, 1:2) = A2(1:50, 1:2);
173             trainingSet(1:50, 3) = 2 * ones(50, 1);
174 
175             testSet(51:100, 1:2) = A2(51:100, :);
176             testSet(51:100, 3) = 2*ones(50, 1);
177             trainingSet(51:100, 1:2) = A1(51:100, 1:2);
178             trainingSet(51:100, 3) = ones(50, 1);
179 
180             gcf = figure("visible", "off");
181             scatter(testSet(:, 1), testSet(:, 2), "fill");
182             hold on;
183             scatter(trainingSet(:, 1), trainingSet(:, 2), "fill")
                    ;
184             grid on;
185             title("Zbior uczacy i testowy");
186             xlabel("c_1");
187             ylabel("c_2");
188             legend("Zbior testowy", "Zbior uczacy");
189             saveas(gcf, "images/test_training.png");
190         end
```

```
191
192         function [A1, A2] = getClasses(obj, set)
193             j = 1;
194             k = 1;
195             A1 = [];
196             A2 = [];
197             for i = 1 : length(set)
198                 if (set(i, 3) == 1)
199                     A1(j, 1 : 2) = set(i, 1 : 2);
200                     j = j + 1;
201                 else
202                     A2(k, 1 : 2) = set(i, 1 : 2);
203                     k = k + 1;
204                 end
205             end
206         end
207     end
208 end
209
210 % DLDAM1 =    75.2800
211 % DLDAM2 =    71.200
212 % LDA    =    75.5400
213 % QDA    =    91.6
```

```
1  classdef ACC < handle
2      properties
3          tp
4          tn
5          fp
6          fn
7          acc
8      end
9
10     methods
11         function obj = ACC()
12             obj.tp = 0;
13             obj.tn = 0;
14             obj.fp = 0;
15             obj.fn = 0;
16             obj.acc = 0;
17         end
18
19         function calculate(obj, class, test)
20             if (test(:, 3) == 2)
21                 if (class > 0)
22                     obj.fp = obj.fp + 1;
23                 else
24                     obj.tn = obj.tn + 1;
25                 end
```

```
26              else
27                  if (class > 0)
28                      obj.tp = obj.tp + 1;
29                  else
30                      obj.fn = obj.fn + 1;
31                  end
32              end
33              obj.acc = (obj.tp + obj.tn) / (obj.tp + obj.tn + obj.
                    fp + obj.fn) * 100;
34          end
35
36
37          function calculateQDA(obj, class, test)
38              if (test(:, 3) == 2)
39                  if (class ~= 2)
40                      obj.fp = obj.fp + 1;
41                  else
42                      obj.tn = obj.tn + 1;
43                  end
44              else
45                  if (class ~= 2)
46                      obj.tp = obj.tp + 1;
47                  else
48                      obj.fn = obj.fn + 1;
49                  end
50              end
51              obj.acc = (obj.tp + obj.tn) / (obj.tp + obj.tn + obj.
                    fp + obj.fn) * 100;
52          end
53      end
54  end
```